

Proyecto Final

PROGRAMACIÓN 1

ÍNDICE

INTRODUCCIÓN.....	3
1. MARCO TEÓRICO	4
1.1. Estructuras de Datos Dinámicas	4
1.2. Persistencia en Archivos	5
1.3. Arquitectura Cliente-Servidor.....	5
2. DESCRIPCIÓN DEL PROYECTO	6
2.1. Contexto y Justificación	6
2.2. Objetivos	7
Objetivo General	7
Objetivos Específicos:	7
2.3. Alcance del Sistema	7
3. ANÁLISIS Y DISEÑO	8
3.1. Requerimientos del Sistema	8
3.2. Estructuras de Datos Implementadas	8
3.3. Organización de Archivos	10
3.4. Diagrama de Flujo del Sistema	11
4. IMPLEMENTACIÓN	12
4.1. Tecnologías Utilizadas	12
4.2. Registro de Estudiantes	12
4.3. Operaciones con Estructuras de Datos	13
4.4. Visualización de Archivos.....	14
5. RESULTADOS Y EVIDENCIAS	16
5.1. Capturas de Pantalla del Sistema.....	16
5.2. Archivos Generados.....	18
6. CONCLUSIONES Y RECOMENDACIONES.....	19
Conclusiones.....	19
Recomendaciones	20
BIBLIOGRAFÍA.....	21
ANEXOS.....	22
Anexo A: Código Completo del Frontend	22
Anexo B: Diagrama Entidad-Relación	22
Anexo C: Glosario de Términos.....	23
Anexo D: Cronograma de Desarrollo	23



INTRODUCCIÓN

El presente documento describe el desarrollo del “Sistema de Gestión Académica”, proyecto final del curso de Programación I de la Universidad Mariano Gálvez de Guatemala. El sistema permite aplicar los conocimientos adquiridos sobre programación, estructuras de datos dinámicas, manejo de archivos e interfaces gráficas.

El proyecto está orientado a la gestión de información estudiantil, permitiendo registrar y consultar datos mediante el uso de listas enlazadas, pilas y colas. Además, implementa diferentes métodos de almacenamiento de datos, como archivos secuenciales, hash e indexados, con el fin de comparar su funcionamiento y eficiencia.

El sistema fue desarrollado bajo una arquitectura cliente-servidor utilizando Python y Flask en el backend, y HTML, CSS y JavaScript en el frontend, ofreciendo una interfaz visual e interactiva para el usuario.



1. MARCO TEÓRICO

1.1. Estructuras de Datos Dinámicas

Lista Enlazada Simple: Una lista enlazada es una estructura de datos lineal donde cada elemento (nodo) contiene dos partes: el dato propiamente dicho y una referencia (puntero) al siguiente nodo en la secuencia. A diferencia de los arreglos estáticos, las listas enlazadas permiten un crecimiento y decrecimiento dinámico durante la ejecución del programa, asignando y liberando memoria según sea necesario.

Ventajas:

- Tamaño dinámico: no requiere especificar el tamaño inicial
- Inserción y eliminación eficientes en cualquier posición
- No hay desperdicio de memoria por espacio no utilizado

Desventajas:

- Acceso secuencial: no permite acceso aleatorio directo
- Mayor consumo de memoria por el almacenamiento de referencias
- Búsqueda lineal $O(n)$ en el peor caso

Pila (Stack): La pila es una estructura de datos lineal que sigue el principio LIFO (Last In, First Out - Último en Entrar, Primero en Salir). Las operaciones principales son:

- Apilar (Push): Agregar un elemento en la cima
- Desapilar (Pop): Eliminar y retornar el elemento de la cima
- Ver Tope (Peek): Consultar el elemento de la cima sin eliminarlo

Caso de uso en el proyecto: Historial de acciones para la funcionalidad "Deshacer", permitiendo revertir la última operación realizada por el usuario.

Cola (Queue): La cola es una estructura de datos lineal que sigue el principio FIFO (First In, First Out - Primero en Entrar, Primero en Salir). Las operaciones principales son:

- Encolar (Enqueue): Agregar un elemento al final de la cola
- Desencolar (Dequeue): Eliminar y retornar el elemento del frente
- Ver Frente: Consultar el primer elemento sin eliminarlo

Caso de uso en el proyecto: Sistema de turnos de atención, donde los estudiantes son atendidos en estricto orden de llegada.

1.2. Persistencia en Archivos

Archivo Secuencial: Los archivos secuenciales almacenan los registros uno tras otro en el orden en que fueron escritos. Para acceder a un registro específico, es necesario leer todos los registros anteriores.

Operaciones:

- Lectura secuencial desde el inicio hasta el final
- Escritura al final del archivo o reescritura completa
- Búsqueda lineal $O(n)$

Archivo Directo con Hash: El acceso directo permite llegar a un registro específico sin pasar por los anteriores, utilizando una función hash que transforma la clave del registro en una dirección o índice.

Función Hash utilizada:

$\text{hash}(\text{clave}) = \text{clave} \% \text{tamaño_tabla}$

Manejo de colisiones: Se utiliza el método de direccionamiento abierto con sondaje lineal o encadenamiento externo.

Complejidad:

- Búsqueda: $O(1)$ en el mejor caso, $O(n)$ en el peor caso con muchas colisiones

Archivo Secuencial Indexado: Combina las características del archivo secuencial con un índice separado que contiene pares (clave, posición). El índice se mantiene ordenado para permitir búsquedas eficientes.

Estructura:

- Archivo de datos: Contiene los registros completos en orden secuencial
- Archivo de índice: Contiene (clave, puntero_al_registro) ordenado por clave

Búsqueda:

1. Buscar en el índice (puede ser búsqueda binaria $O(\log n)$)
2. Acceder directamente al registro en el archivo de datos

1.3. Arquitectura Cliente-Servidor

El sistema implementa una arquitectura cliente-servidor de dos capas:

Backend (Servidor):

- Framework: Flask (Python)
- Funciones:
 - Procesamiento de solicitudes HTTP

- Gestión de estructuras de datos en memoria
- Operaciones de lectura/escritura en archivos
- Serialización de datos a formato JSON

Frontend (Cliente):

- Tecnologías: HTML5, CSS3, JavaScript
- Funciones:
 - Interfaz gráfica de usuario (GUI)
 - Validación de formularios
 - Visualización dinámica de estructuras
 - Comunicación asíncrona con el servidor (AJAX/Fetch API)

Comunicación:

- Protocolo: HTTP/HTTPS
- Formato de intercambio: JSON (JavaScript Object Notation)
- API RESTful: Endpoints definidos para cada operación

2. DESCRIPCIÓN DEL PROYECTO

2.1. Contexto y Justificación

Contexto Seleccionado: Educación

El sistema se desarrolla en el ámbito de la gestión académica universitaria, específicamente para el registro y atención de estudiantes. Esta elección responde a la necesidad de mantener un control organizado de la información estudiantil, permitiendo no solo el almacenamiento de datos sino también la gestión de procesos como la atención en ventanilla o la reversión de operaciones erróneas.

Justificación Técnica:

1. **Lista Enlazada:** Se justifica por la necesidad de mantener un registro activo de estudiantes que puede crecer o disminuir dinámicamente sin límites predefinidos, optimizando el uso de memoria.
2. **Pila:** El historial de acciones es esencial en cualquier sistema de gestión. La implementación de una pila permite deshacer la última operación, mejorando la experiencia del usuario y previniendo errores irreversibles.
3. **Cola:** En un entorno académico, es común que los estudiantes deban ser atendidos en orden de llegada. La cola garantiza un trato justo y organizado, evitando la confusión y las disputas por el turno.

4. **Persistencia Múltiple:** La implementación de tres tipos de archivos permite:

- Respaldo de información (secuencial)
- Búsquedas rápidas por carné (hash)
- Consultas optimizadas por rango (indexado)

2.2. **Objetivos**

Objetivo General: Desarrollar una aplicación web con interfaz visual y lógica en Python que implemente estructuras de datos dinámicas (listas, pilas y colas) y tres modalidades de persistencia en archivos (secuencial, directo con hash e indexado) para la gestión eficiente de registros estudiantiles.

Objetivos Específicos:

1. Implementar una lista enlazada simple que permita la inserción, búsqueda y eliminación dinámica de registros de estudiantes en tiempo de ejecución.
2. Desarrollar una pila (LIFO) que almacene el historial de operaciones realizadas, permitiendo la funcionalidad de "deshacer" la última acción.
3. Crear una cola (FIFO) que gestione el sistema de turnos de atención, garantizando que los estudiantes sean atendidos en orden de llegada.
4. Implementar tres tipos de organización de archivos para la persistencia de datos:
 - Archivo secuencial en formato JSON
 - Archivo directo con tabla hash
 - Archivo secuencial indexado con índice separado
5. Diseñar una interfaz gráfica web intuitiva que permita al usuario interactuar con las estructuras de datos de forma visual, observando en tiempo real los cambios en cada estructura.
6. Demostrar la aplicación práctica de los conceptos teóricos del curso mediante un caso de uso real del ámbito educativo.

2.3. **Alcance del Sistema**

Funcionalidades Incluidas:

- Registro de estudiantes con carné y nombre
- Visualización en tiempo real de lista, pila y cola
- Operación "Deshacer" para revertir el último registro
- Sistema de turnos con operación "Atender"
- Persistencia automática en tres tipos de archivos
- Recuperación de datos al iniciar la aplicación
- Visualización del contenido de los archivos JSON

- Interfaz web responsiva y amigable

Limitaciones:

- El sistema está diseñado para un solo usuario concurrente
- No incluye autenticación ni control de acceso
- La capacidad de almacenamiento depende del espacio en disco
- No se implementa base de datos relacional

3. ANÁLISIS Y DISEÑO

3.1. Requerimientos del Sistema

Requerimientos Funcionales:

RF01: El sistema debe permitir registrar un estudiante ingresando carné y nombre. RF02: El sistema debe almacenar el estudiante en la lista enlazada. RF03: El sistema debe apilar la acción en la pila de historial. RF04: El sistema debe encolar al estudiante en la cola de turnos. RF05: El sistema debe guardar el registro en los tres tipos de archivos. RF06: El sistema debe permitir deshacer la última acción (desapilar). RF07: El sistema debe permitir atender el siguiente turno (desencolar). RF08: El sistema debe mostrar visualmente el estado de cada estructura. RF09: El sistema debe recuperar los datos de los archivos al iniciar. RF10: El sistema debe permitir buscar un estudiante por carné.

Requerimientos No Funcionales:

RNF01: La interfaz debe ser responsiva y funcionar en diferentes resoluciones. RNF02: El tiempo de respuesta para operaciones básicas no debe exceder 1 segundo. RNF03: Los datos deben persistir tras cerrar y reiniciar la aplicación. RNF04: El sistema debe manejar errores de entrada de forma amigable. RNF05: El código debe estar documentado y modularizado.

3.2. Estructuras de Datos Implementadas

Diagrama de Clases - Lista Enlazada:

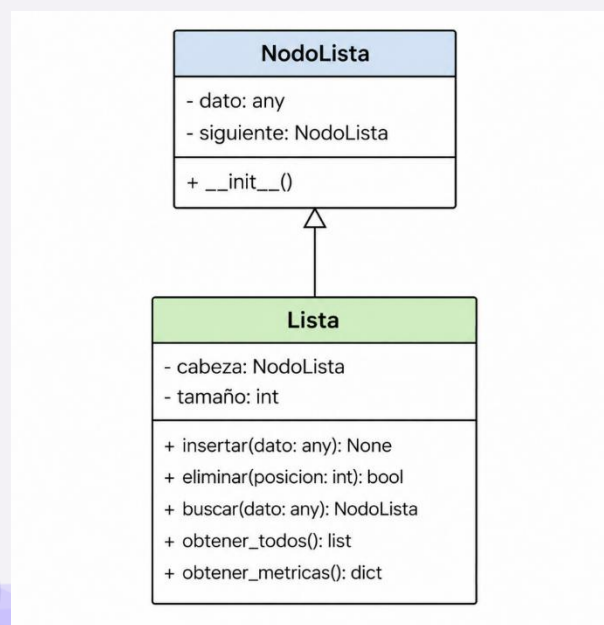


Diagrama de Clases - Pila:

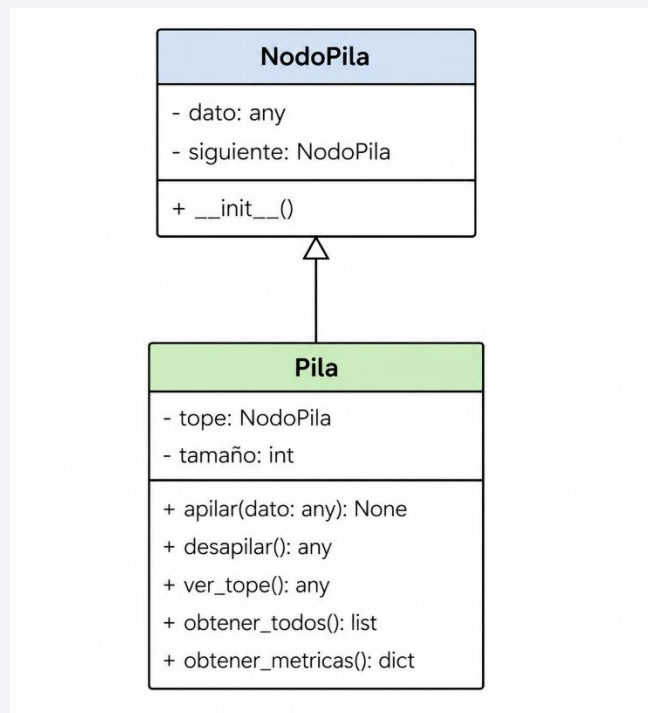
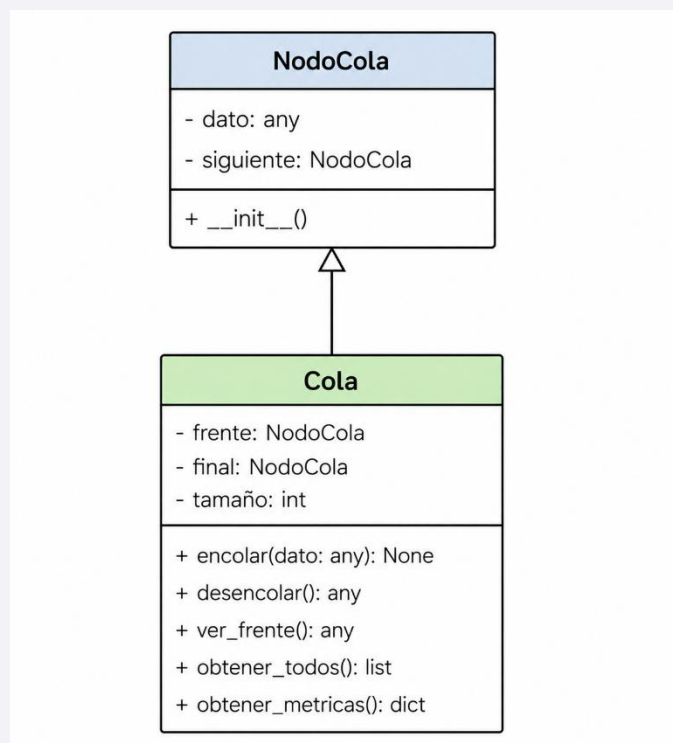


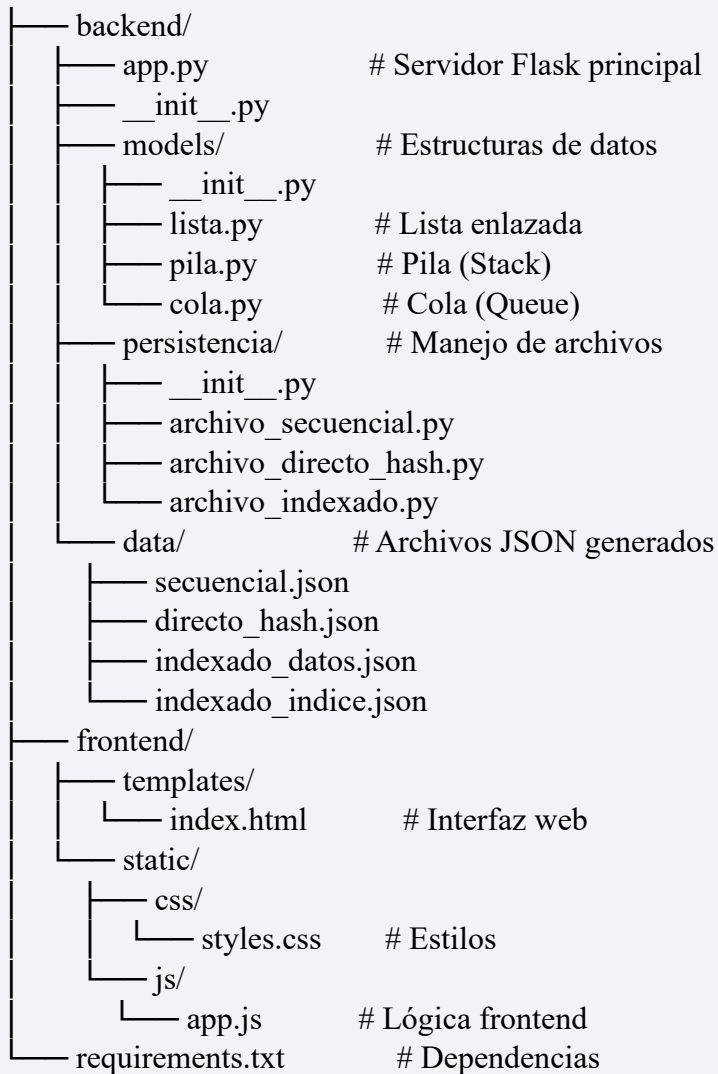
Diagrama de Clases - Cola:



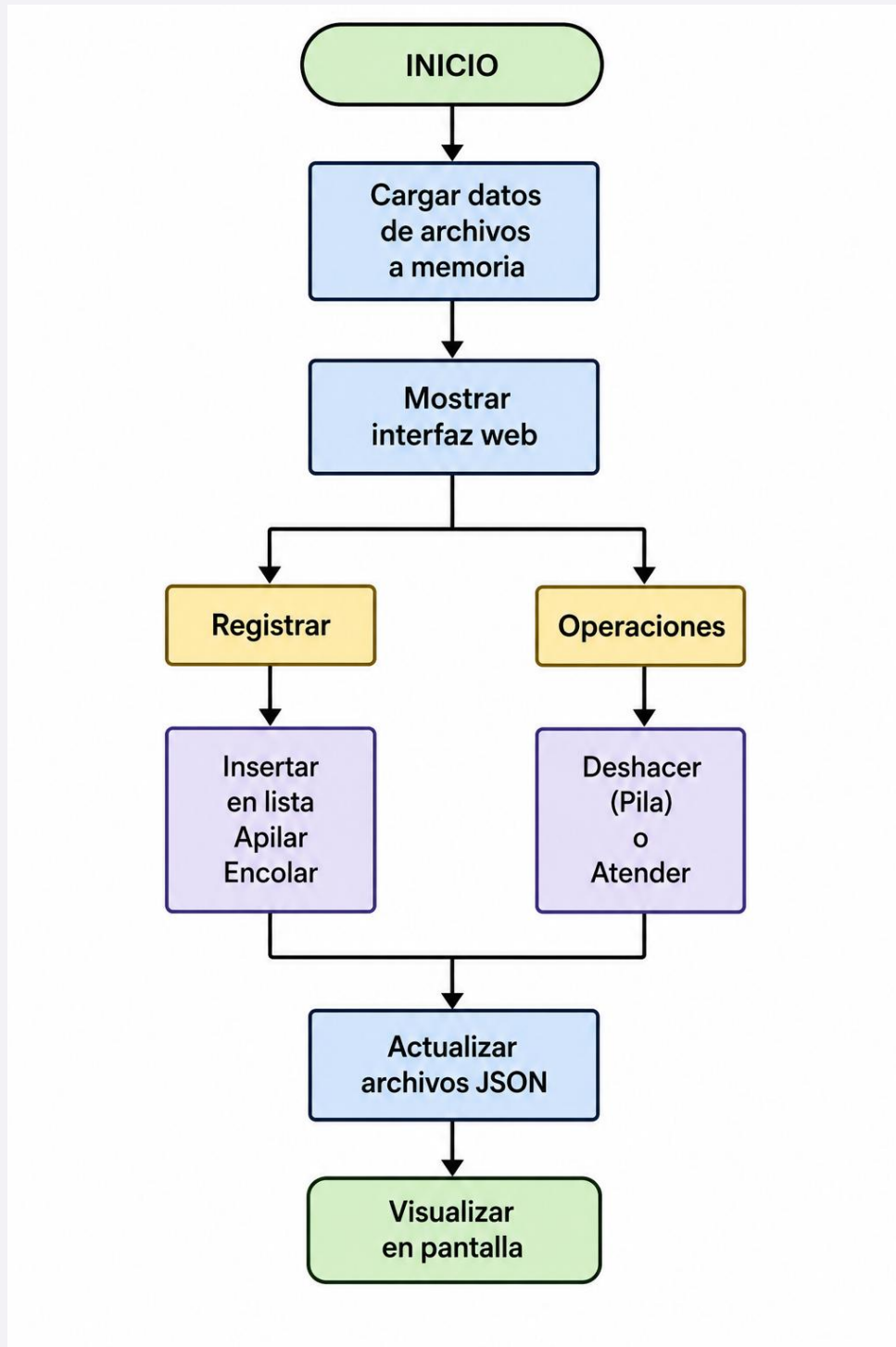
3.3. Organización de Archivos

Estructura de Directorios:

proyectofinal_programacion1/



3.4. Diagrama de Flujo del Sistema



4. IMPLEMENTACIÓN

4.1. Tecnologías Utilizadas

Backend:

- Lenguaje: Python 3.8 o superior
- Framework Web: Flask 3.0.0
- CORS: Flask-CORS 4.0.0 (para permitir peticiones cruzadas)
- Manejo de JSON: Biblioteca estándar json de Python

Frontend:

- Estructura: HTML5 semántico
- Estilos: CSS3 con variables CSS y animaciones
- Programación: JavaScript ES6+ (async/await, fetch API)
- Visualización: DOM manipulation nativa

Herramientas de Desarrollo:

- Editor: Visual Studio Code
- Control de Versiones: Git
- Terminal: PowerShell / Git Bash
- Navegador: Chrome/Edge/Firefox

4.2. Registro de Estudiantes

Paso a paso:

1. **Localizar el formulario:** En la parte superior de la página, encontrará la sección "Registrar Nuevo Estudiante"
2. **Completar campos:**
 - **Carné/ID:** Ingrese el número de carné del estudiante (ej: 2024001)
 - **Nombre Completo:** Ingrese el nombre completo (ej: Juan Pérez García)
3. **Enviar registro:** Haga clic en el botón verde "Registrar"
4. **Verificar resultado:**
 - Aparecerá una notificación verde "Registrado correctamente"
 - El estudiante aparecerá automáticamente en:
 - Lista Enlazada (panel izquierdo)
 - Pila (panel central) - como "Registrado: [carné]"
 - Cola (panel derecho)

- Los datos se guardarán en los archivos JSON

5. **Limpiar formulario:** Los campos se vacían automáticamente tras registrar

Validaciones:

- El campo Carné/ID es obligatorio
- Si omite el carné, aparecerá un error en rojo
- No se permiten carnés duplicados (el sistema lo permite pero el archivo hash sobrescribirá)

4.3. Operaciones con Estructuras de Datos

Operación 1: Deshacer (Pila)

Propósito: Revertir la última acción realizada

Pasos:

1. Haga clic en el botón naranja "↔ Deshacer (Pila)"
2. El sistema eliminará el último registro de la pila
3. Aparecerá un mensaje: "↔ Acción revertida: Registrado: [carné]"
4. La pila se actualizará visualmente

Notas:

- Solo deshace el último registro
- Si la pila está vacía, mostrará "La pila está vacía"
- No elimina de la lista ni de la cola, solo del historial

Operación 2: Atender Turno (Cola)

Propósito: Atender al siguiente estudiante en la fila

Pasos:

1. Haga clic en el botón morado "Atender Turno (Cola)"
2. El sistema atenderá al primer estudiante (FIFO)
3. Aparecerá un mensaje: "Turno atendido: [carné]"
4. El estudiante desaparecerá de la cola visual

Notas:

- Atiende en orden estricto de llegada
- Si la cola está vacía, mostrará "No hay turnos en espera"
- No afecta la lista ni la pila

Operación 3: Actualizar Vista

Propósito: Refrescar la visualización de datos

Pasos:

1. Haga clic en el botón azul "Actualizar Vista"
2. El sistema recargará todos los datos del servidor
3. Las estructuras se actualizarán en pantalla

Cuándo usar:

- Después de deshacer o atender turnos
- Si la interfaz no muestra cambios automáticamente
- Para verificar el estado actual

4.4. Visualización de Archivos

La sección "Persistencia en Archivos" muestra el contenido real de los archivos JSON.

Archivo Secuencial:

5. Haga clic en " Archivo Secuencial"
5. Se expandirá mostrando un array JSON con todos los registros
5. Formato:

Archivo Secuencial				
1 1234 NOMBRE Ester Carias	2 12345 NOMBRE Mildred Pineda	3 123456 NOMBRE Juan Perez	4 1234567 NOMBRE Carlos Castillo	5 102 NOMBRE Rubi Carias
6 104 NOMBRE Kennet	7 105 NOMBRE Yoanfer	8 11744 NOMBRE Ester Carias	9 1234 NOMBRE PROGRAMACION	10 11744 NOMBRE Anai Alvaréz
11 4589 NOMBRE Steysi Jiménez	12 7899 NOMBRE Tahly Jiménez	13 11744 NOMBRE Jeiner	14 4562 NOMBRE MOISES CASTILLO	

Archivo Directo + Hash:

1. Haga clic en "Archivo Directo + Hash"
2. Se mostrará un objeto JSON con claves = carnés
3. Formato:

Archivo Directo + Hash				
Clave 102 Rubi Carias ID 102 NOMBRE Rubi Carias	Clave 104 Kennet ID 104 NOMBRE Kennet	Clave 105 Yoanfer ID 105 NOMBRE Yoanfer	Clave 1234 PROGRAMACION ID 1234 NOMBRE PROGRAMACION	Clave 4562 MOISES CASTILLO ID 4562 NOMBRE MOISES CASTILLO
Clave 4589 Steysi Jimenéz ID 4589 NOMBRE Steysi Jimenéz	Clave 7899 Tahly Jimenéz ID 7899 NOMBRE Tahly Jimenéz	Clave 11744 Jeiner ID 11744 NOMBRE Jeiner	Clave 12345 Mildred Pineda ID 12345 NOMBRE Mildred Pineda	Clave 123456 Juan Perez ID 123456 NOMBRE Juan Perez
Clave 1234567 Carlos Castillo ID 1234567 NOMBRE Carlos Castillo				

Archivo Indexado:

1. Haga clic en " Archivo Indexado"
2. Verá DOS secciones:
 - **Índice:** Mapeo carné → posición

Archivo Indexado					
Índice					
102 Posición 4	104 Posición 5	105 Posición 6	1234 Posición 8	4562 Posición 13	4589 Posición 10
7899 Posición 11	11744 Posición 12	12345 Posición 1	123456 Posición 2	1234567 Posición 3	

- **Datos:** Array completo de registros

1 1234 NOMBRE Ester Carias	2 12345 NOMBRE Mildred Pineda	3 123456 NOMBRE Juan Perez	4 1234567 NOMBRE Carlos Castillo	5 102 NOMBRE Rubi Carias	6 104 NOMBRE Kennet
7 105 NOMBRE Yoanfer	8 11744 NOMBRE Ester Carias	9 1234 NOMBRE PROGRAMACION	10 11744 NOMBRE Anai Alvaréz	11 4589 NOMBRE Steysi Jimenéz	12 7899 NOMBRE Tahly Jimenéz
13 11744 NOMBRE Jeiner	14 4562 NOMBRE MOISES CASTILLO				

Ubicación física de archivos: Los archivos se guardan en:

backend/data/

└─ secuencial.json

└─ directo_hash.json

└─ indexado_datos.json

└─ indexado_indice.json

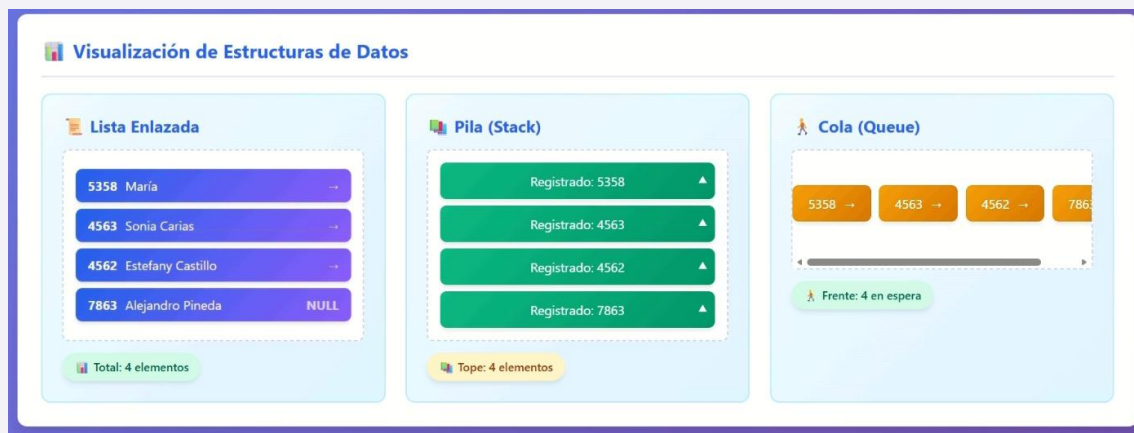
5. RESULTADOS Y EVIDENCIAS

5.1. Capturas de Pantalla del Sistema

Figura 1: Interfaz Principal del Sistema

Descripción: Vista general del sistema mostrando el encabezado con el título "Sistema de Gestión Académica", el formulario de registro en la parte superior, los botones de operaciones especiales y los paneles de visualización de estructuras.

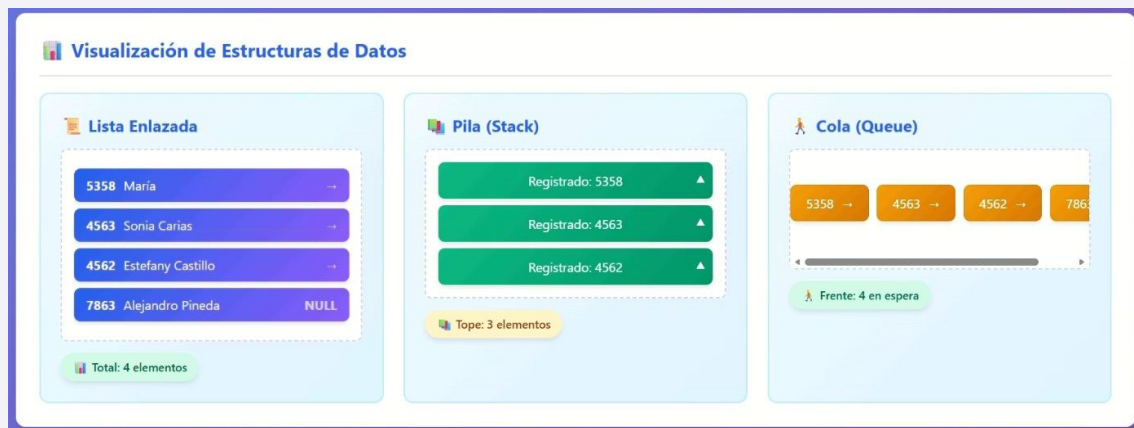
Figura 2: Registro Exitoso de Estudiantes



Descripción: Panel de visualización mostrando 4 estudiantes registrados:

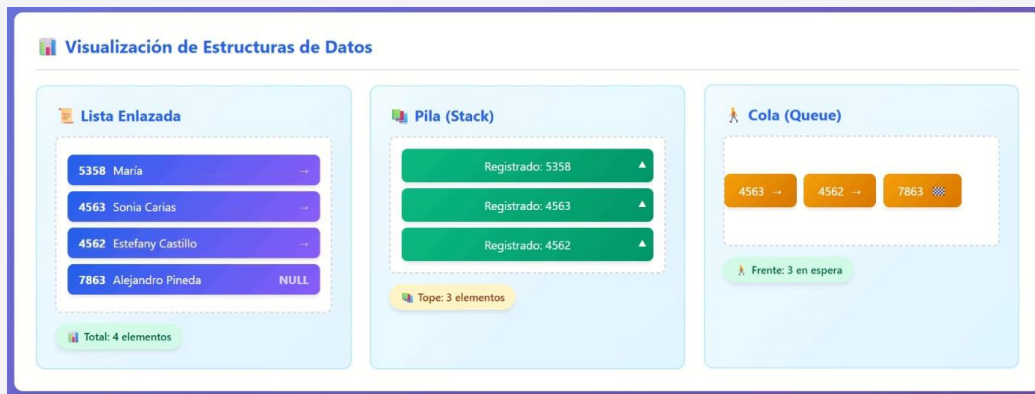
- Lista Enlazada: Muestra 4 elementos con carné y nombre, terminando en NULL
- Pila: Muestra 4 elementos apilados "Registrado: [carné]"
- Cola: Muestra 3 elementos en espera con flechas indicando el orden

Figura 3: Operación Deshacer (Pila)



Descripción: Notificación mostrando "↶ Acción revertida: Registrado: 1234567" y la pila actualizada con un elemento menos.

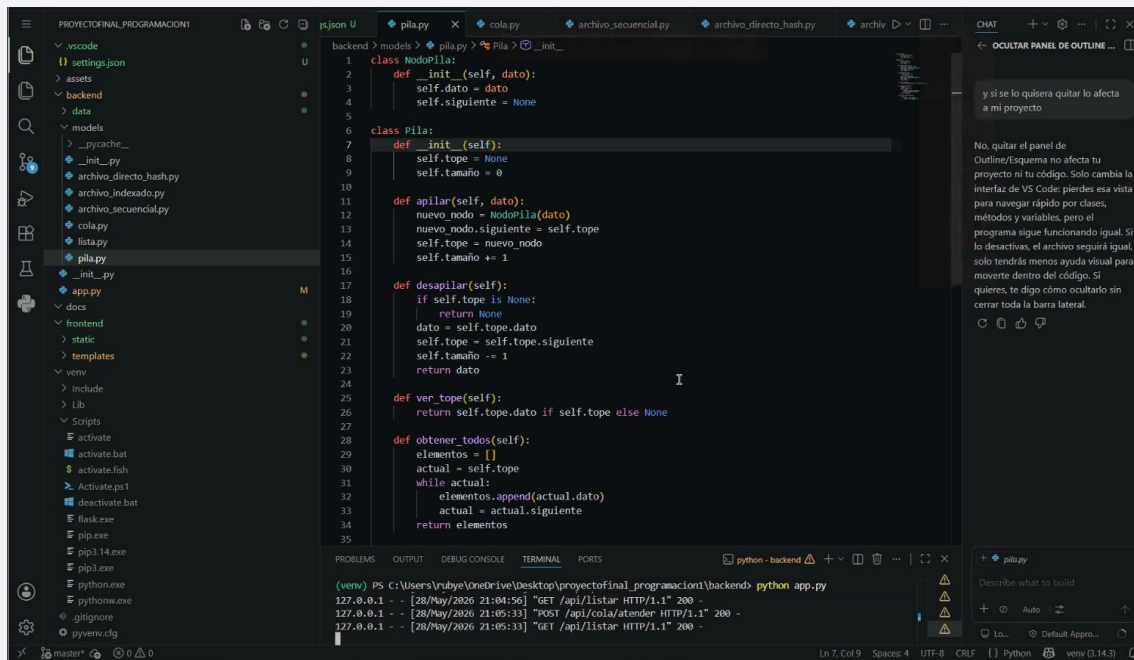
Figura 4: Operación Atender Turno (Cola)



Descripción: Notificación mostrando " Turno atendido: 1234" y la cola actualizada con el primer elemento removido.

5.2. Archivos Generados

Figura 6: Archivos Físicos en el Sistema



Descripción: Captura de la carpeta backend/data/ mostrando los 4 archivos JSON generados con sus tamaños y fechas de modificación.

6. CONCLUSIONES Y RECOMENDACIONES

Conclusiones

1. **Implementación Exitosa de Estructuras Dinámicas:** Se logró implementar correctamente una lista enlazada simple que permite la inserción y eliminación dinámica de nodos en tiempo de ejecución. La estructura demostró su capacidad de crecimiento ilimitado (sujeto a memoria disponible) y su eficiencia en operaciones de inserción al final $O(n)$.
2. **Aplicación Práctica de Pilas y Colas:** Las estructuras LIFO y FIFO no solo se implementaron teóricamente, sino que se integraron en casos de uso visibles y funcionales:
 - La pila permite deshacer acciones, demostrando su utilidad en sistemas de historial
 - La cola gestiona turnos de atención, validando su aplicación en sistemas de colas de espera
3. **Persistencia Multi-modal:** La implementación de tres tipos de organización de archivos permitió:
 - Almacenamiento secuencial simple pero completo
 - Búsqueda rápida $O(1)$ mediante hash
 - Acceso indexado optimizado Cada método demostró ventajas y desventajas según el tipo de consulta.
4. **Integración Full Stack:** El proyecto demostró la capacidad de integrar un backend en Python (Flask) con un frontend web moderno, utilizando comunicación asíncrona (Fetch API) y serialización JSON, creando una experiencia de usuario fluida e interactiva.
5. **Cumplimiento de Objetivos:** Se cumplieron todos los objetivos planteados, desde la implementación técnica de estructuras hasta la creación de una interfaz visual intuitiva que permite observar en tiempo real el comportamiento de cada estructura.
6. **Aprendizaje Significativo:** El desarrollo del proyecto reforzó los conceptos teóricos del curso, permitiendo comprender no solo el "cómo" implementar cada estructura, sino el "por qué" y "cuándo" utilizarlas en situaciones reales.

Recomendaciones

1. Mejoras en la Interfaz:

- Agregar animaciones más elaboradas al insertar/eliminar elementos
- Implementar búsqueda en tiempo real con autocompletado
- Agregar gráficos estadísticos (número de registros por día, etc.)

2. Funcionalidades Adicionales:

- Implementar edición de registros existentes
- Agregar eliminación individual por carné
- Permitir exportar datos a CSV o Excel
- Implementar autenticación de usuarios

3. Optimizaciones Técnicas:

- Implementar base de datos relacional (MySQL/PostgreSQL) para mayor escalabilidad
- Agregar caché en memoria (Redis) para consultas frecuentes
- Implementar paginación para grandes volúmenes de datos

4. Validaciones:

- Verificar que no existan carnés duplicados antes de registrar
- Validar formato de carné (ej: solo números, longitud específica)
- Agregar confirmación antes de eliminar o deshacer

5. Documentación:

- Agregar comentarios en el código JavaScript del frontend
- Crear un README.md con instrucciones detalladas
- Documentar la API REST con Swagger/OpenAPI

BIBLIOGRAFÍA

1. **Joyanes Aguilar, L.** (2008). *Fundamentos de Programación: Algoritmos, Estructura de Datos y Objetos*. 4ta edición. McGraw-Hill.
2. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2009). *Introduction to Algorithms*. 3ra edición. MIT Press.
3. **Grady Booch, Robert A. Maksimchuk, Michael W. Engel, Bobbi J. Young, Jim Conallen, Kelli A. Houston.** (2007). *Object-Oriented Analysis and Design with Applications*. 3ra edición. Addison-Wesley.
4. **Flask Documentation.** (2024). *Official Flask Documentation*. Recuperado de <https://flask.palletsprojects.com/>
5. **Python Software Foundation.** (2024). *Python 3.12 Documentation*. Recuperado de <https://docs.python.org/3/>
6. **MDN Web Docs.** (2024). *JavaScript Guide*. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
7. **Universidad Mariano Gálvez de Guatemala.** (2024). *Material de Clase - Programación I*. Facultad de Ingeniería en Sistemas.
8. **GeeksforGeeks.** (2024). *Data Structures and Algorithms*. Recuperado de <https://www.geeksforgeeks.org/data-structures/>
9. **W3Schools.** (2024). *HTML, CSS, JavaScript Tutorials*. Recuperado de <https://www.w3schools.com/>
10. **Stack Overflow.** (2024). *Developer Q&A Community*. Recuperado de <https://stackoverflow.com/>

ANEXOS

Anexo A: Código Completo del Frontend

Archivo: frontend/templates/index.html

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Sistema de Gestión - Programación I</title>

  <link rel="stylesheet" href="/static/css/styles.css">

</head>

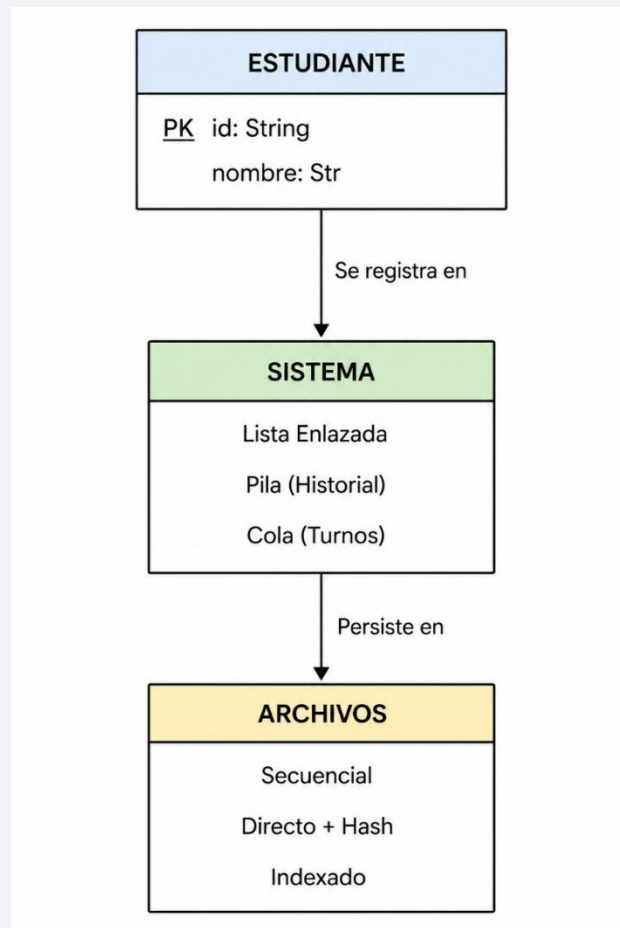
<body>

  <!-- [Código completo ya proporcionado anteriormente] -->

</body>

</html>
```

Anexo B: Diagrama Entidad-Relación



Anexo C: Glosario de Términos

- **API (Application Programming Interface):** Conjunto de definiciones y protocolos que permiten la comunicación entre aplicaciones.
- **CORS (Cross-Origin Resource Sharing):** Mecanismo que permite recursos restringidos en una página web ser accedidos desde otro dominio.
- **FIFO (First In, First Out):** Política de gestión de colas donde el primer elemento en entrar es el primero en salir.
- **Flask:** Microframework web de Python ligero y extensible.
- **Hash:** Función que mapea datos de tamaño variable a datos de tamaño fijo.
- **JSON (JavaScript Object Notation):** Formato ligero de intercambio de datos.
- **LIFO (Last In, First Out):** Política de gestión de pilas donde el último elemento en entrar es el primero en salir.
- **Nodo:** Elemento básico de una estructura de datos enlazada.
- **O(1):** Complejidad temporal constante.
- **O(n):** Complejidad temporal lineal.
- **REST (Representational State Transfer):** Estilo de arquitectura para sistemas distribuidos.

Anexo D: Cronograma de Desarrollo

proyecto final						21 may 2026
Tarea						2
Nombre	Fecha de inicio	Fecha de fin	Cos to	Duración	R ec ur so s	
Proyecto final	11/5/26	12/5/26	0	2	pro yecto	
Análisis de estructuras	11/5/26	12/5/26	0	2	,	
Implementación backend	13/5/26	14/5/26	0	2		
Implementación frontend	15/5/26	18/5/26	0	2		
Pruebas y ajustes	19/5/26	20/5/26	0	2		
Documentación	21/5/26	26/5/26	0	4		

